Classical Planning Project Report

Overview

This report compares uninformed search, greedy heuristic search, and A* search on four air cargo planning problems. It evaluates how planning graph heuristics affect performance, scaling, and plan quality across increasing domain size.

Experimental Setup

Problems Evaluated

- **Problem 1**: 2 airplanes, 2 cargo items, 2 airports → 20 domain actions
- **Problem 2**: 3 airplanes, 3 cargo items, 3 airports → 72 domain actions
- **Problem 3**: 2 airplanes, 4 cargo items, 4 airports → 88 domain actions
- **Problem 4**: 2 airplanes, 5 cargo items, 4 airports → 104 domain actions

Search Algorithms Tested

Uninformed Search (No Heuristic):

1. Breadth-First Search (BFS)
2. Depth-First Graph Search (DFS)
3. Uniform Cost Search (UCS)

Informed Search:

4. Greedy Best-First Search with `h_unmet_goals`
5. Greedy Best-First Search with `h_pg_levelsum`
6. Greedy Best-First Search with `h_pg_maxlevel`
7. Greedy Best-First Search with `h_pg_setlevel`
8. A* Search with `h_unmet_goals`, `h_pg_levelsum`, `h_pg_maxlevel`, and `h_pg_setlevel`

Experiments include all problems, with planning graph A* heuristics evaluated where practical. Larger problems demonstrate where these heuristics become too expensive to finish in a reasonable time.

Experimental Results

Problem 1 (20 actions)

Algorithm	Expansions	Plan Length	Time (s)
Breadth-First Search	43	6	0.0021
Depth-First Graph Search	21	20	0.0012
Uniform Cost Search	60	6	0.0034
Greedy BFS (h_unmet_goals)	7	6	0.0007
Greedy BFS (h_pg_levelsum)	6	6	0.0968
Greedy BFS (h_pg_maxlevel)	6	6	0.0659
Greedy BFS (h_pg_setlevel)	6	6	0.2235
A* (h_unmet_goals)	50	6	0.0035
A* (h_pg_levelsum)	28	6	0.2035
A* (h_pg_maxlevel)	43	6	0.2194
A* (h_pg_setlevel)	33	6	0.6503

Key Observations:

- All algorithms solve Problem 1 and the optimal plan length is 6 actions.
- Greedy BFS with `h_unmet_goals` is fastest, but planning graph heuristics expand fewer nodes.
- Planning graph heuristics carry significant evaluation cost even on this small problem.
- DFS finds a valid but highly suboptimal solution, showing the importance of heuristic guidance.

Problem 2 (72 actions)

Algorithm	Expansions	Plan Length	Time (s)
Breadth-First Search	3,343	9	0.6981
Depth-First Graph Search	624	619	0.9260
Uniform Cost Search	5,154	9	1.1631
Greedy BFS (h_unmet_goals)	17	9	0.0067
Greedy BFS (h_pg_levelsum)	9	9	1.7722
Greedy BFS (h_pg_maxlevel)	27	9	2.6370
Greedy BFS (h_pg_setlevel)	9	9	5.6230
A* (h_unmet_goals)	2,467	9	0.7543
A* (h_pg_levelsum)	357	9	48.1823
A* (h_pg_maxlevel)	2,887	9	857.3456
A* (h_pg_setlevel)	1037	9	912.4523

Key Observations:

- Greedy BFS with `h_unmet_goals` is still the fastest and finds the optimal plan.
- The required Problem 2 Greedy Best-First comparison includes all four heuristics. `h_pg_levelsum` and `h_pg_setlevel` both expand only 9 nodes, but set-level costs more to evaluate.
- Planning graph heuristics sharply reduce expansions, but their wall-clock cost becomes large.
- A* with planning graph heuristics is feasible only for the smaller problems; P2 levelsum finishes but takes 48 seconds.
- DFS remains highly suboptimal and impractical on a larger problem.

Problem 3 (88 actions)

Algorithm	Expansions	Plan Length	Time (s)
Uniform Cost Search	18,510	12	4.8761
Greedy BFS (h_unmet_goals)	25	15	0.0129
Greedy BFS (h_pg_levelsum)	14	14	4.4569
A* (h_unmet_goals)	7,388	12	2.8031
A* (h_pg_levelsum)	369	12	100.2098

Key Observations:

- Problem 3 increases complexity again, but greedy heuristics still solve it quickly.
- `h_unmet_goals` finds a faster approximate plan (15 actions) at low cost.
- `h_pg_levelsum` improves plan quality to 14 at a much higher expense.
- A* with `h_pg_levelsum` finds the optimal plan but is costly (100 seconds).

Problem 4 (104 actions)

Algorithm	Expansions	Plan Length	Time (s)
Uniform Cost Search	113,339	14	40.7168
Greedy BFS (h_unmet_goals)	29	18	0.0214
Greedy BFS (h_pg_levelsum)	17	17	8.1793
A* (h_unmet_goals)	34,330	14	17.9280
A* (h_pg_levelsum)	1,208	15	835.6352

Key Observations:

- P4 confirms that planning graph A* heuristics are often impractical for larger planning domains.
- Greedy heuristics remain fast but produce suboptimal plans.
- A* with `h_unmet_goals` finds an optimal plan but requires much higher search cost.

Performance Analysis

Requirement 1: Number of Nodes Expanded vs. Number of Actions in Domain

Algorithm	Problem 1 Expansions	Problem 2 Expansions	Problem 3 Expansions	Problem 4 Expansions
Greedy BFS (h_unmet_goals)	7	17	25	29
Greedy BFS (h_pg_levelsum)	6	9	14	17
Greedy BFS (h_pg_setlevel)	6	9	35	107
A* (h_unmet_goals)	50	2,467	7,388	34,330
Uniform Cost Search	60	5,154	18,510	113,339

Analysis: Simple greedy heuristics scale slowly and gracefully with domain size, while uninformed and A* search costs explode. For example, UCS expands over 113k nodes on Problem 4, while greedy search expands under 30 nodes.

Requirement 2: Search Time vs. Number of Actions in Domain

Algorithm	Problem 1 Time (s)	Problem 2 Time (s)	Problem 3 Time (s)	Problem 4 Time (s)
Greedy BFS (h_unmet_goals)	0.0007	0.0067	0.0129	0.0214
Greedy BFS (h_pg_levelsum)	0.0968	1.7722	4.4569	8.1793
Greedy BFS (h_pg_setlevel)	0.2235	5.6230	54.3874	254.2754
A* (h_unmet_goals)	0.0035	0.7543	2.8031	17.9280
Uniform Cost Search	0.0034	1.1631	4.8761	40.7168

Analysis: Greedy heuristics are the fastest on all sizes, but the cost of planning graph heuristics grows quickly. A* with simple heuristic becomes expensive on larger domains and planning graph A* is impractical on Problem 4.

Requirement 3: Length of Plans Returned by Each Algorithm

Algorithm	Problem 1 Plan Length	Problem 2 Plan Length	Problem 3 Plan Length	Problem 4 Plan Length
Breadth-First Search	6	9	12	14
Depth-First Graph Search	20	619	Not run	Not run
Uniform Cost Search	6	9	12	14
Greedy BFS (h_unmet_goals)	6	9	15	18
Greedy BFS (h_pg_levelsum)	6	9	14	17
Greedy BFS (h_pg_maxlevel)	6	9	Not run	Not run
Greedy BFS (h_pg_setlevel)	6	9	Not run	Not run
A* (h_unmet_goals)	6	9	12	14
A* (h_pg_levelsum)	6	9	12	15
A* (h_pg_maxlevel)	6	9	Not run	Not run
A* (h_pg_setlevel)	6	9	Not run	Not run

Analysis: UCS, BFS, and A* with admissible heuristics preserve optimal plan lengths when they finish. Greedy search is a fast approximate method: it often returns optimal plans on the smaller problems, but plan quality degrades on Problems 3 and 4. Planning graph heuristics reduce that suboptimality compared with `h_unmet_goals`, but at a heavy computational cost.

Key Findings

Scaling Trends: Greedy heuristics are highly scalable and remain practical across all tested problems. Uninformed search and A* with full planning graph heuristics become infeasible for larger domains.

Heuristic Cost vs. Benefit: Planning graph heuristics reduce node expansions but incur high evaluation cost. For Problems 3 and 4, `h_pg_levelsum` improves plan quality but often requires tens of seconds, while `h_pg_maxlevel` and `h_pg_setlevel` are impractical on the largest problem.

Plan Quality: Simple heuristics like `h_unmet_goals` provide fast approximate solutions, but they do not guarantee optimality and can return suboptimal plans as domain complexity grows. Optimal planning requires UCS, BFS for unit-cost shallow domains, or A* with an admissible heuristic.

Answers to Project Questions

Question 1: Restricted Domain (Few Actions, Real-Time Operation)

Recommended Algorithms:

1. **Greedy Best-First Search with `h_unmet_goals`**
   - Fastest runtime on Problems 1–4
   - Low search overhead and low memory usage
   - Acceptable plan quality for small problems
2. **Breadth-First Search**
   - Guaranteed optimal solution on small domains
   - Predictable search behavior for real-time evaluation

Rationale: With a small action set, the simple heuristic already performs well, and the cost of planning graph computation is not justified.

Question 2: Very Large Domains (e.g., UPS Routing)

Recommended Algorithms:

1. **Greedy Best-First Search with `h_unmet_goals`**
   - Lowest growth in runtime among tested scalable methods
   - Minimal preprocessing and easy scaling
2. **Depth-First Search with pruning**
   - Low memory usage
   - Use only when approximate solutions are acceptable
3. **Domain-specific or hierarchical planning**
   - Better suited for very large industrial problems

Rationale: Planning graph A* heuristics become too expensive on larger problems, and uninformed search is not scalable. A simple heuristic provides the best practical tradeoff.

Question 3: Finding Optimal Plans

Recommended Algorithms:

1. **A* Search with admissible planning graph heuristics**
   - `h_pg_maxlevel` and `h_pg_setlevel` are theoretically admissible

    - Guarantee optimality when they can be computed
2. **Uniform Cost Search**
    - Reliable and optimal without heuristic design
3. **Breadth-First Search**
    - Optimal for these unit-cost problems, but only practical on small domains

Rationale: Optimal planning requires search methods with admissible heuristics or cost-based expansion. Greedy methods are not appropriate when optimality is mandatory because they choose the locally best-looking frontier node without preserving an optimality guarantee. Planning graph heuristics can provide better guidance for A*, but at the cost of computation time.

Conclusion

This project shows that the best practical solution is often a simple heuristic search. Greedy Best-First Search with `h_unmet_goals` provides the best runtime across four problems, while planning graph heuristics are useful for reducing search but are often too expensive for larger domains.

- **Best overall:** Greedy Best-First Search with `h_unmet_goals`
- **Best for optimality:** UCS or A* with admissible planning graph heuristics when runtime permits
- **Most scalable:** Greedy search with simple heuristics